

MaxHeap Email Priority Queue – GenAI Comparison

1. Objective

The program simulates a CEO's email inbox. It stores emails in a list-based MaxHeap, placing the highest-priority sender category first. When two emails have the same sender category, the newer date is used as the tie-breaker. **NEXT** displays the next email without removing it, **READ** removes it, and **COUNT** reports how many emails remain.

Two GenAI programs were compared: Gemini and GPT-5.6 Luna. Both programs were generated in C from the same assignment requirements.

2. Requirements

Program Requirements

- ☐ Use C
- ☐ Implement a MaxHeap
- ☐ Use a list-based implementation
- ☐ Create heap functions from scratch
- ☐ Prioritize emails by sender category
- ☐ Use the newest email first when sender categories are the same
- ☐ Implement **EMAIL**
- ☐ Implement **NEXT**
- ☐ Implement **READ**
- ☐ Implement **COUNT**
- ☐ Handle an empty queue
- ☐ Handle multiple **NEXT** commands
- ☐ Handle multiple **READ** commands

Email Priorities

1. Boss
2. Subordinate
3. Peer
4. ImportantPerson
5. OtherPerson

GenAI Access and Shared Prompt

How I accessed the programs: I used Gemini through its web interface and GPT-5.6 Luna through the ChatGPT web interface. I copied the assignment requirements into each tool rather than changing the request between tools.

Prompt used for both GenAIs:

Use two GenAI's to generate C code to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.): •

The program will prioritize emails for a busy company CEO. • You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status. • You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO. • You must create functions from scratch. Do not include pre-existing heap modules. • Test File format: Command Description EMAIL , , The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume is one of the five strings listed above. Assume is a string which may contain spaces, but not commas Assume is in the format: MM-DD-YYYY NEXT Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date: READ CEO has read email and dealt with it COUNT Display current count of unread emails • The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it). • Sample Test File: EMAIL Peer,Can you help me on this?,12-01-2024 EMAIL OtherPerson,Try our product,12-19-2024 EMAIL Boss,Important,12-20-2024 EMAIL Subordinate,How do I handle this?,12-25-2024 EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024 EMAIL Boss,Never Mind,01-03-2025 COUNT NEXT READ NEXT READ COUNT • Your program should display this: There are 6 emails to read. Next email: Sender: Boss Subject: Never Mind Date: 01-03-2025 Next email: Sender: Boss Subject: Important Date: 12-20-2024 There are 4 emails to read.

3. GenAI #1

GenAI Used: Gemini

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure to represent an email
typedef struct {
    char sender[32];
    char subject[256];
    char date[11]; // MM-DD-YYYY
    int priority_score; // Higher score = higher priority
} Email;

// Structure for the Priority Queue (MaxHeap using dynamic list)
```

```
typedef struct {
    Email *data;
    int capacity;
    int size;
} MaxHeap;

// Helper: Convert MM-DD-YYYY to an integer YYYYMMDD for easy date comparison
int parse_date(const char *date_str) {
    int month, day, year;
    sscanf(date_str, "%d-%d-%d", &month, &day, &year);
    return year * 10000 + month * 100 + day;
}

// Helper: Convert sender category to base priority level (higher number = higher priority)
int get_sender_priority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    if (strcmp(sender, "OtherPerson") == 0) return 1;
    return 0;
}

// Compare two emails: Returns > 0 if email 'a' has higher priority than 'b'
int compare_emails(const Email *a, const Email *b) {
    int prio_a = get_sender_priority(a->sender);
    int prio_b = get_sender_priority(b->sender);

    // Priority 1: Sender Category
    if (prio_a != prio_b) {
        return prio_a - prio_b;
    }

    // Priority 2: Newest email first (Tie-breaker using Date)
    int date_a = parse_date(a->date);
    int date_b = parse_date(b->date);
    return date_a - date_b;
}

// Initialize MaxHeap
MaxHeap* create_heap(int initial_capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = initial_capacity;
    heap->size = 0;
    heap->data = (Email*)malloc(sizeof(Email) * heap->capacity);
    return heap;
}

// Swap helper
void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}
```

```

}

// Heapify Up (used during insertion)
void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(&heap->data[index], &heap->data[parent]) > 0) {
            swap(&heap->data[index], &heap->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

// Heapify Down (used during extraction)
void heapify_down(MaxHeap *heap, int index) {
    int max_idx = index;
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;

        if (left < heap->size && compare_emails(&heap->data[left], &heap->data[max_idx]) > 0) {
            max_idx = left;
        }
        if (right < heap->size && compare_emails(&heap->data[right], &heap->data[max_idx]) > 0) {
            max_idx = right;
        }

        if (max_idx != index) {
            swap(&heap->data[index], &heap->data[max_idx]);
            index = max_idx;
        } else {
            break;
        }
    }
}

// Push an email to MaxHeap
void heap_push(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email*)realloc(heap->data, sizeof(Email) * heap->capacity);
    }
    heap->data[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

// Pop (Remove and return) the highest-priority email
int heap_pop(MaxHeap *heap, Email *output) {
    if (heap->size == 0) return 0; // Empty

```

```
*output = heap->data[0];
heap->data[0] = heap->data[heap->size - 1];
heap->size--;
heapify_down(heap, 0);
return 1;
}

// Peek at highest-priority email without removing
int heap_peek(MaxHeap *heap, Email *output) {
    if (heap->size == 0) return 0; // Empty
    *output = heap->data[0];
    return 1;
}

// Free allocated memory
void free_heap(MaxHeap *heap) {
    free(heap->data);
    free(heap);
}

int main() {
    MaxHeap *inbox = create_heap(10);
    char line[512];

    while (fgets(line, sizeof(line), stdin)) {
        // Strip trailing newline/carriage return
        line[strcspn(line, "\r\n")] = 0;
        if (strlen(line) == 0) continue;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            Email e;
            char *payload = line + 6;

            // Parse comma-delimited fields: <sender>,<subject>,<date>
            char *sender = strtok(payload, ",");
            char *subject = strtok(NULL, ",");
            char *date = strtok(NULL, ",");

            if (sender && subject && date) {
                // Trim leading/trailing whitespace if any
                while (*sender == ' ') sender++;
                while (*subject == ' ') subject++;
                while (*date == ' ') date++;

                strncpy(e.sender, sender, sizeof(e.sender) - 1);
                strncpy(e.subject, subject, sizeof(e.subject) - 1);
                strncpy(e.date, date, sizeof(e.date) - 1);

                heap_push(inbox, e);
            }
        }
        else if (strcmp(line, "COUNT") == 0) {
            printf("There are %d emails to read.\n", inbox->size);
        }
    }
}
```

```

        else if (strcmp(line, "NEXT") == 0) {
            Email top;
            if (heap_peek(inbox, &top)) {
                printf("Next email:\n");
                printf("Sender: %s\n", top.sender);
                printf("Subject: %s\n", top.subject);
                printf("Date: %s\n", top.date);
            }
        }
        else if (strcmp(line, "READ") == 0) {
            Email read_email;
            heap_pop(inbox, &read_email);
        }
    }

    free_heap(inbox);
    return 0;
}

```

Correctness

Compiles? - yes Max Heap? - yes Sender categories prioritized correctly? - yes Newer emails prioritized correctly? - yes Does **NEXT** display without removing? - yes Does **READ** remove the highest-priority email? - yes Does **COUNT** display the correct number? - yes Does it handle empty queues? - yes

Observations:

The Gemini program correctly defines a MaxHeap and separates the main operations into helper functions. Its comparison first checks sender priority and then converts each date to **YYYYMMDD**, so a newer date is considered larger. **NEXT** uses **heap_peek** and therefore does not remove the email. **READ** uses **heap_pop**, and **COUNT** reads the current heap size. Both peek and pop safely return failure for an empty queue.

There are several correctness risks in the generated version. **strncpy** does not explicitly add a null terminator when an input exactly fills a destination array, and **Email e** is not initialized before copying. The program does not check whether **malloc** or **realloc** succeeds. It also accepts malformed dates because **sscanf** is not checked, and the input parser only removes leading spaces. The heap logic itself is correct, but these input and memory cases need attention before submission.

Execution Time

Insert an email? - $O(\log n)$ Remove an email? - $O(\log n)$ **NEXT**? - $O(1)$ **COUNT**? - $O(1)$

Observations:

For n emails, insertion takes $O(\log n)$ time because the new item can move from a leaf to the root. Removing the maximum also takes $O(\log n)$ time because the replacement item can move down the heap. **NEXT** is $O(1)$ because it only examines the root. **COUNT** is $O(1)$ because the current size is stored in the heap. Date conversion and sender comparison are constant-time for the fixed-size fields. Reading m input lines takes $O(m)$ overall, excluding output costs.

Space Complexity

- How much memory does the MaxHeap use?
- Is the list/array size fixed or dynamic?
- Are there any unnecessary data structures?

Observations:

On a typical 64-bit system, one Gemini **Email** uses about 304 bytes: 32 bytes for the sender, 256 bytes for the subject, 11 bytes for the date, and padding for the integer field. The initial array holds 10 emails, using about 3,040 bytes, and the **MaxHeap** structure uses about 24 more bytes for its pointer, capacity, and size. Therefore, the initial heap uses about 3,064 bytes, not including the small input buffer and temporary variables. When the array doubles, its memory also doubles. The heap stores n **Email** structures, so its space complexity is $O(n)$.

Maintainability

- Is the code easy to read?
- Are the functions clearly separated?
- Are variable and function names understandable?
- Are there useful comments?
- Would the code be easy to modify later?

Observations:

The code has a useful separation between parsing, comparison, heap movement, insertion, removal, and display. Names such as **heapify_up**, **heapify_down**, **heap_push**, and **heap_pop** clearly describe their jobs, and the comments explain the heap operations. The main maintainability concerns are inconsistent error handling, repeated string parsing during comparisons, lack of input-length validation, and the missing explicit string terminators. These are localized problems and can be fixed without changing the public command behavior.

4. GenAI #2

GenAI Used: GPT-5.6 Luna

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define SUBJECT_SIZE 500

typedef struct {
    char sender[30];
    char subject[SUBJECT_SIZE];
```

```
    char date[11];
    int priority;
} Email;

typedef struct {
    Email emails[MAX_EMAILS];
    int size;
} MaxHeap;

/* Return the priority based on sender category */
int getPriority(char sender[]) {
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1;
}

/*
 * Returns 1 if email a should be ahead of email b.
 * Higher sender priority comes first.
 * If priorities are equal, newer date comes first.
 */
int higherPriority(Email a, Email b) {
    if (a.priority > b.priority)
        return 1;

    if (a.priority < b.priority)
        return 0;

    /*
     * Dates are MM-DD-YYYY, so convert them into
     * YYYYMMDD for easy comparison.
     */
    int aMonth, aDay, aYear;
    int bMonth, bDay, bYear;

    sscanf(a.date, "%d-%d-%d", &aMonth, &aDay, &aYear);
    sscanf(b.date, "%d-%d-%d", &bMonth, &bDay, &bYear);

    if (aYear > bYear)
        return 1;
    if (aYear < bYear)
        return 0;

    if (aMonth > bMonth)
        return 1;
    if (aMonth < bMonth)
        return 0;
```

```
        if (aDay > bDay)
            return 1;

        return 0;
    }

    /* Swap two emails */
    void swapEmails(Email *a, Email *b) {
        Email temp = *a;
        *a = *b;
        *b = temp;
    }

    /* Move an email upward in the heap */
    void heapifyUp(MaxHeap *heap, int index) {
        if (index == 0)
            return;

        int parent = (index - 1) / 2;

        if (higherPriority(heap->emails[index], heap->emails[parent])) {
            swapEmails(&heap->emails[index], &heap->emails[parent]);
            heapifyUp(heap, parent);
        }
    }

    /* Move an email downward in the heap */
    void heapifyDown(MaxHeap *heap, int index) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int highest = index;

        if (left < heap->size &&
            higherPriority(heap->emails[left], heap->emails[highest])) {
            highest = left;
        }

        if (right < heap->size &&
            higherPriority(heap->emails[right], heap->emails[highest])) {
            highest = right;
        }

        if (highest != index) {
            swapEmails(&heap->emails[index], &heap->emails[highest]);
            heapifyDown(heap, highest);
        }
    }

    /* Add an email to the MaxHeap */
    void insert(MaxHeap *heap, Email email) {
        if (heap->size >= MAX_EMAILS)
            return;
    }
}
```

```
    heap->emails[heap->size] = email;
    heap->size++;

    heapifyUp(heap, heap->size - 1);
}

/* Remove the highest-priority email */
Email removeMax(MaxHeap *heap) {
    Email removed = heap->emails[0];

    heap->size--;

    if (heap->size > 0) {
        heap->emails[0] = heap->emails[heap->size];
        heapifyDown(heap, 0);
    }

    return removed;
}

/* Display the highest-priority email without removing it */
void nextEmail(MaxHeap *heap) {
    if (heap->size == 0)
        return;

    printf("Next email:\n");
    printf("Sender: %s\n", heap->emails[0].sender);
    printf("Subject: %s\n", heap->emails[0].subject);
    printf("Date: %s\n", heap->emails[0].date);
}

/* Main program */
int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <test_file>\n", argv[0]);
        return 1;
    }

    FILE *file = fopen(argv[1], "r");

    if (file == NULL) {
        printf("Could not open file.\n");
        return 1;
    }

    MaxHeap heap;
    heap.size = 0;

    char line[600];

    while (fgets(line, sizeof(line), file) != NULL) {

        /* Remove newline */
        line[strcspn(line, "\n")] = '\0';
```

```
/* Process EMAIL command */
if (strncmp(line, "EMAIL ", 6) == 0) {
    Email email;

    char *data = line + 6;

    /*
     * Split the line into:
     * sender, subject, date
     */
    char *sender = strtok(data, ",");
    char *subject = strtok(NULL, ",");
    char *date = strtok(NULL, ",");

    if (sender != NULL && subject != NULL && date != NULL) {
        strcpy(email.sender, sender);
        strcpy(email.subject, subject);
        strcpy(email.date, date);

        email.priority = getPriority(email.sender);

        insert(&heap, email);
    }
}

/* Process NEXT command */
else if (strcmp(line, "NEXT") == 0) {
    nextEmail(&heap);
}

/* Process READ command */
else if (strcmp(line, "READ") == 0) {
    if (heap.size > 0) {
        removeMax(&heap);
    }
}

/* Process COUNT command */
else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap.size);
}
}

fclose(file);

return 0;
}
```

Correctness

Does the code compile? - yes Does it correctly implement a MaxHeap? - yes Does it prioritize the sender categories correctly? - yes Does it correctly prioritize newer emails? - yes Does **NEXT** display the highest-priority email without removing it? - yes Does **READ** remove the highest-priority email? - yes Does **COUNT** display the correct number? - yes Does it handle empty queues correctly? - yes

Observations:

The GPT program also implements the heap operations correctly for valid input. Its **higherPriority** function compares sender priority and then year, month, and day, so the ordering rule is correct. **NEXT**, **READ**, and **COUNT** have the required behavior, and **READ** checks that the heap is nonempty before removing an item.

The main interface difference is that GPT expects a test-file path as a command-line argument, while Gemini reads commands from standard input. That may be inconvenient if the assignment's driver sends commands through standard input. GPT also has important safety problems: **strcpy** can overflow every string field, the input line can be longer than the fixed buffer, **removeMax** assumes the heap is nonempty, and the fixed array silently rejects emails after 1,000 entries. Unknown sender categories are treated as **OtherPerson**, which can hide invalid input. The program does check whether the input file opens successfully.

Execution Time

Insert an email? - $O(\log n)$ Remove an email? - $O(\log n)$ **NEXT**? - $O(1)$ **COUNT**? - $O(1)$

Observations:

Insertion and removal are both $O(\log n)$ because the heap can move an item along its height. **NEXT** and **COUNT** are $O(1)$. GPT's date comparison performs a fixed number of operations, so it does not change these asymptotic bounds. Its recursive heapify functions use $O(\log n)$ call-stack space in the worst case.

Space Complexity

On a typical 64-bit system, one GPT **Email** uses about 548 bytes: 30 bytes for the sender, 500 bytes for the subject, 11 bytes for the date, the priority integer, and structure padding. The fixed array of 1,000 emails uses about 548,000 bytes, plus about 4 bytes for the heap size. The input buffer adds 600 bytes, so the program uses approximately 548,604 bytes before other small stack and runtime overhead. Its space usage is fixed at $O(1000)$ and it reserves this memory even when the inbox is nearly empty.

Maintainability

GPT has clear function names and concise comments, but its naming style is inconsistent with the Gemini version (**getPriority**, **heapifyUp**, and **heapifyDown** use camel case). Passing complete **Email** structures by value to **higherPriority** copies large character arrays unnecessarily. The command-line file interface is explicit, but it is less reusable with standard-input test harnesses. The fixed-size and unsafe string operations also make future changes riskier.

5. Comparison and Selection

Both programs have the same heap asymptotic operation costs: $O(\log n)$ for insertion and removal, and $O(1)$ for **NEXT** and **COUNT**. Gemini is preferable for this assignment because it uses a dynamically sized list,

reads the command stream from standard input, handles empty **NEXT** and **READ** operations through return values, and uses pointer parameters instead of copying full emails during comparisons. GPT is shorter in some places, but its fixed 1,000-email limit and **strcpy** calls create more serious correctness and maintainability risks.

I selected the Gemini program as the basis for the assignment. This choice is based on compatibility with the command-oriented input format, dynamic storage, and clearer separation of heap operations. The selection is about the generated design; the safety issues listed above must still be fixed in the final source before it is submitted.

6. Improvements Identified for the Selected Program

The following changes are the improvements I identified for the Gemini version. They are documented here for implementation in the source code:

- **Correctness:** initialize each **Email**, validate that all three fields are present, reject unknown sender categories and malformed dates, trim field whitespace, and explicitly terminate copied strings.
- **Execution time:** retain the MaxHeap because it already gives $O(\log n)$ insertion and removal, $O(1)$ **NEXT**, and $O(1)$ **COUNT**. Avoid scanning the entire inbox for any command. A parsed numeric date could also be stored in the email so comparisons do not repeatedly parse the same date.
- **Space complexity:** retain dynamic doubling so storage remains $O(n)$ without the fixed 1,000-email limit. The input line and one temporary email remain $O(1)$ additional space.
- **Maintainability:** keep the heap operations in separate functions, use consistent names, use bounded copies, check allocation results, and make invalid-input behavior explicit.
- **Input** The final version also takes in a test file like the assignment requires. The previous AI editions were not asked to do this so I didn't mention it in my comparison but for the purposes of the assignment the Final version would be more correct.